

[TÊN CƠ QUAN CHỦ QUẢN]

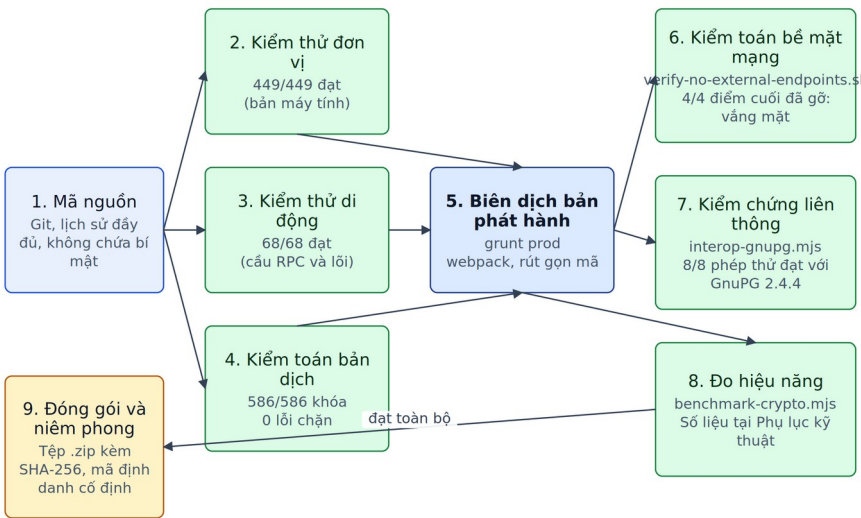
[TÊN ĐƠN VỊ]

PHỤ LỤC KỸ THUẬT

PHỤ LỤC KỸ THUẬT

Toàn văn kết quả kiểm thử, đo đạc và kiểm toán — kèm hướng dẫn tái lập

Quy trình xây dựng và kiểm chứng một bản phát hành



Các bước 6, 7 và 8 đều là kịch bản chạy được, kết quả kèm trong hồ sơ. Người thẩm định chạy lại được trên chính bản phát hành để tự kiểm chứng, không phải tin vào lời khẳng định.

Thuộc hồ sơ

Phiên bản sản phẩm

Ngày thực hiện đo

Môi trường đo

Sáng kiến Secure Mail

Secure Mail v0.3.0

08/9/2026

AMD EPYC 4 nhân, 7,8 GB RAM,

Linux

[Địa danh], tháng 9 năm 2026

MỤC LỤC

...

(Mở tệp trong Word và nhấn Ctrl+A rồi F9 để cập nhật mục lục và số trang)

MỤC ĐÍCH CỦA PHỤ LỤC

Phụ lục này chứa **toàn văn kết quả** của mọi phép đo và phép thử được viện dẫn trong Bản thuyết minh, cùng **lệnh chính xác** để chạy lại từng phép.

Mục đích không phải là trưng bày số liệu mà là để Hội đồng, hoặc bất kỳ ai được giao thẩm định, **tự kiểm chứng thay vì phải tin tác giả**. Trong an toàn thông tin, một khẳng định không kiểm chứng được thì không có giá trị.

Toàn bộ kết quả dưới đây được sinh ra trong cùng một lần chạy, ngày 08/9/2026, trên bản phát hành `secure-mail-v0.3.0-hardened`.

A. KIỂM THỬ VÀ KIỂM TOÁN

A.1. Môi trường đo

Bảng A.1. Môi trường thực hiện các phép đo

Thành phần	Giá trị
Bộ xử lý	AMD EPYC Processor (with IBPB), 4 nhân
Bộ nhớ	7,8 GB
Hệ điều hành	Linux 6.8.0-139-generic
Node.js	v22.23.1 (đo hiệu năng, kiểm thử) và v24.15.0 (biên dịch bản phát hành)
Trình duyệt dùng để chụp và kiểm thử giao diện	Chrome for Testing 149.0.7827.22
Thư viện mật mã	OpenPGP.js 5.11.3 — đúng phiên bản mà bản phát hành đóng gói
Phần mềm đối chứng liên thông	GnuPG 2.4.4, libgcrypt 1.10.3

Số liệu hiệu năng đặc trưng cho **cách hiện thực của phần mềm**, không đặc trưng cho phần cứng của một triển khai cụ thể. Cấu hình máy được ghi lại để người đọc quy chiếu.

A.2. Kiểm thử đơn vị bản máy tính

Lệnh chạy lại:

```
cd
npx jest --selectProjects unit
```

mailvelope

Kết quả:

Test Suites:	22	passed,	22	total
Tests:		449	passed,	449
Snapshots:			0	total
Time:			31.977	s
Ran all test suites.				

Ba phép thử trong bộ này đã thất bại trước khi hoàn thiện hồ sơ, do vẫn kiểm tra tên và địa chỉ của sản phẩm gốc sau khi đổi nhận diện sản phẩm:

- `ActionMenuWrapper` › should render help link with correct href — vẫn mong đợi `https://www.mailvelope.com/faq`
- `ActionMenuWrapper` › should render action menu header with logo — vẫn mong đợi `img/Mailvelope/logo.svg`
- `KeyServer` › should render learn more links — vẫn đếm liên kết tới `mailvelope.com/faq`

Đây là nợ kỹ thuật thực sự: một bộ kiểm thử không chạy trọn vẹn thì mất giá trị bảo đảm, vì lỗi mới sẽ lẫn vào những lỗi đã biết. Cả ba đã được sửa để kiểm tra đúng tài nguyên của Secure Mail.

A.3. Kiểm thử bản di động**Lệnh chạy lại:**

<code>cd</code>	<code>mobile</code>
<code>npx jest</code>	

Kết quả:

Test Suites:	10	passed,	10	total
Tests:		68	passed,	68
Snapshots:			0	total
Time:			36.65	s
Ran all test suites in 3 projects.				

Nội dung 68 phép thử bao gồm: hợp đồng cầu RPC có kiểu (định dạng gói tin, timeout, lan truyền lỗi sập), các lớp đệm thay thế API của Chrome, các bộ xử lý chòm khóa, mã hóa, sao lưu và tùy chọn, **phép thử đối chứng (golden)** so khớp kết quả với dữ liệu chuẩn sinh từ bản máy tính, và **phép thử liên thông** giải mã bằng thư viện độc lập.

A.3.1. Kiểm chứng bản máy tính không bị thay đổi

Ràng buộc "không được sửa bản máy tính đang chạy ổn định trong khi phát triển bản di động" được biến thành một phép kiểm tra tự động.

Lệnh chạy lại:

```
cd mobile
./scripts/verify-desktop-untouched.sh
```

Kết quả:

```
OK: mailvelope/ pristine at e85da308 (161 extension files verified)
```

Kịch bản kiểm tra ba điều: cây làm việc của kho bản máy tính sạch, HEAD trùng với mốc đã ghim, và 161 tệp trong build/chrome/ khớp mã băm SHA-256 của bản phát hành.

Trong quá trình xây dựng hồ sơ này, kịch bản đã **từ chối** ngay khi bản máy tính được sửa để gỡ bỏ thư viện đo lường và cơ chế kiểm tra giấy phép — đúng như thiết kế. Mốc ghim chỉ được cập nhật cho những thay đổi đã được chủ động quyết định, và mỗi lần cập nhật đều được ghi lại trong chính kịch bản.

A.4. Kiểm toán bản dịch tiếng Việt

Lệnh chạy lại:

```
cd mailvelope
node scripts/verify-translation.mjs
```

Kết quả:

```
English      keys      :      586
Vietnamese   keys      :      586
Coverage     :      100%
Missing      :      0
Placeholder  errors:      0
Markup       errors      :      0
Untranslated      :      2
Glossary      deviations:      0

Other locales: {"ar":531,"de":579,"en":586,"es":489,"fr":534,"he":507,
               "id":531,"ja":528,"km":512,"lt":517,"my":512,"pt_BR":531,"ru":530,
               "tr":533,"uk":534,"vi":586}
```

Hai giá trị được báo là "chưa dịch" là Gmail API và Copyright © Tracy Tran — đều là danh từ riêng, giữ nguyên là đúng. Không có sai lệch thuật ngữ nào.

A.4.1. Sáu lỗi do kiểm toán phát hiện và đã sửa

Ghi lại đầy đủ, vì việc một quy trình tự tìm ra lỗi là bằng chứng quy trình đó hoạt động.

Bảng A.2. Các lỗi bản dịch được phát hiện và cách xử lý

Mã chuỗi	Lỗi	Mức độ	Xử lý
onboarding_success_created_key_text	Bản dịch bỏ mất câu yêu cầu gửi khóa công khai và dấu vân tay cho quản trị viên qua kênh đã xác thực — tức bỏ mất chính bước bảo đảm tính xác thực của khóa	Ảnh hưởng an toàn	Dịch lại đầy đủ cả ba câu
recovery_sheet_trusted_contacts	Bản dịch bỏ mất khuyến cáo phải tự xác minh danh tính đối với liên hệ dùng nhà cung cấp thư khác	Ảnh hưởng an toàn	Dịch lại đầy đủ
recovery_sheet_key_server	Mất hai trong ba ô thay thế \$1, làm thông báo hiển thị thiếu giá trị động	Lỗi chức năng	Khôi phục đủ ba ô
7 chuỗi khác nhau	Thuật ngữ "private key" được dịch không nhất quán: lẫn giữa "khóa bí mật" và "khóa riêng tư"	Ảnh hưởng nhận thức	Thống nhất về "khóa riêng tư"
onboarding_setup_alert	Từ "fingerprint" để nguyên tiếng Anh, không nhất quán với bộ thuật ngữ	Chất lượng	"dấu vân tay (fingerprint)"
decrypt_digital_signature_sender_mismatch_tooltip	Diễn đạt "khóa đã ký" không nói rõ là khóa công khai của người ký	Chất lượng	Diễn đạt lại chính xác

Lý do chọn "khóa riêng tư" thay vì "khóa bí mật": tránh gây liên tưởng nhầm tới "bí mật nhà nước", đồng thời nhấn vào tính sở hữu cá nhân của khóa — người dùng phải hiểu rằng đây là thứ **của riêng mình** và không bao giờ được giao cho ai, kể cả quản trị viên.

A.5. Kiểm toán bề mặt mạng của gói phát hành

Lệnh chạy lại:

```
cd mailvelope \
./scripts/verify-no-external-endpoints.sh
releases/secure-mail-v0.3.0-hardened/extension
```

Kết quả (toàn văn):

```
Secure Mail — network-surface audit of
releases/secure-mail-v0.3.0-hardened/extension
=====
[1] Forbidden endpoints (must be absent)
-----
PASS cleaninsights 0 occurrences
PASS metrics.cleaninsights.org 0 occurrences
PASS license.mailvelope.com 0 occurrences
PASS mailvelope.com/google-workspace 0 occurrences
[2] Network call sites in the source (complete list)
-----
src/controller/app.controller.js
src/lib/svg-file-parser.js
src/modules/gmail.js
src/modules/mveloKeyServer.js
src/modules/openpgpKeyServer.js
src/modules/wkdLocate.js
Resolved hosts and their status:
accounts.google.com Gmail sign-in ENABLED (feature in
use)
oauth2.googleapis.com OAuth token exchange ENABLED (feature in
use)
www.googleapis.com Gmail API + userinfo ENABLED (feature in
use)
keys.mailvelope.com public key server DISABLED by default
keys.openpgp.org public key server DISABLED by default
<recipient domain> WKD key lookup by domain DISABLED by default
chrome-extension://... bundled SVG assets local only, never
remote
=====
RESULT: PASS — no removed endpoint is present.
```

A.5.1. Ghi chú về phương pháp kiểm toán

Kịch bản cố ý **không** liệt kê mọi chuỗi giống URL tìm thấy trong gói đã biên dịch. Cách làm đó cho ra hàng trăm kết quả sai: các thư viện phía giao diện nhúng liên kết tài liệu trong thông báo lỗi của chúng, và những liên kết đó không bao giờ được truy cập.

Bề mặt mạng thật của một chương trình là **tập hợp các điểm phát sinh yêu cầu mạng**. Kịch bản liệt kê đúng tập hợp đó bằng cách tìm mọi lời gọi fetch, XMLHttpRequest, WebSocket và sendBeacon trong mã nguồn. Kết quả là **sáu tệp**, và trong đó `svg-file-parser.js` chỉ nạp tệp đồ họa đóng gói sẵn qua `chrome.runtime.getURL()` — luôn là địa chỉ nội bộ của tiện ích, không bao giờ ra mạng.

A.5.2. Đối chiếu trước và sau

Bảng A.3. Số lần xuất hiện của các điểm cuối bên thứ ba trong gói đã biên dịch

Chuỗi tìm kiếm	Bản trước cải tạo	Bản v0.3.0
<code>cleaninsights</code>	9	0
<code>license.mailvelope.com</code>	1	0
<code>mailvelope.com/google-workspace</code>	1	0
<code>keys.mailvelope.com</code>	2	2 — mã còn, chức năng tắt mặc định

Dòng cuối được ghi lại **đúng như thực tế** thay vì làm tròn thành số không: mã tra cứu máy chủ khóa vẫn còn trong sản phẩm, chỉ là tắt mặc định. Giữ lại là có chủ đích — nếu về sau đơn vị tự vận hành một máy chủ khóa nội bộ thì bật lại được. Hồ sơ phân biệt rõ hai mức bảo đảm này thay vì gộp chung.

A.6. Kiểm chứng liên thông với GnuPG

Đây là phép thử quan trọng nhất về mặt bảo đảm lâu dài: nó chứng minh bản mã của Secure Mail là **OpenPGP chuẩn**, đọc được bằng phần mềm khác, trên nền tảng khác, kể cả khi Secure Mail không còn được dùng. Nói cách khác, dữ liệu của đơn vị không bị khóa vào một sản phẩm.

Lệnh chạy lại:

```
cd mailvelope
node scripts/interop-gnupg.mjs
```

Kịch bản sinh một cặp khóa RSA-4096 bằng thư viện của Secure Mail và một cặp khóa RSA-4096 bằng GnuPG trong một thư mục khóa tạm, rồi trao đổi thư thật

theo cả hai chiều. Nội dung thử là văn bản tiếng Việt có dấu, nên phép thử đồng thời xác nhận đường xử lý UTF-8 không bị hỏng khi đi qua một phần mềm khác.

Kết quả (toàn văn):

```

pgp          (GnuPG)          2.4.4          /          openpgp          5.11.3

pgp:          key          B1A8FA76FC3F0D74:          public          key
          "Secure Mail User <securemail@donvi.test>"          imported
pgp:          Total          number          processed:          1
pgp:          imported:          1

PASS          Key          import          (Secure Mail -> GnuPG)
          GnuPG accepted the RSA-4096 public key produced by OpenPGP.js
PASS          Key          import          (GnuPG -> Secure Mail)
          OpenPGP.js parsed the GnuPG key, fingerprint D35D0816317DC81A...
PASS          A. Confidentiality          (Secure Mail -> GnuPG)
          GnuPG recovered the plaintext byte-for-byte, diacritics intact
PASS          A. Authenticity          (Secure Mail -> GnuPG)
          GnuPG reported GOODSIG for the Secure Mail signature
PASS          B. Confidentiality          (GnuPG -> Secure Mail)
          OpenPGP.js recovered the plaintext byte-for-byte, diacritics
intact
PASS          B. Authenticity          (GnuPG -> Secure Mail)
          OpenPGP.js verified the GnuPG signature against the imported key
PASS          C. Detached signature          (Secure Mail -> GnuPG)
          GnuPG verified a detached signature produced by Secure Mail
PASS          D. Integrity          (tampered ciphertext)
          A single flipped character in the ciphertext is rejected,
          not silently mis-decrypted

ALL PASS

```

A.6.1. Ý nghĩa của từng phép thử

- **Nhập khóa hai chiều** — định dạng khóa công khai tương thích, đơn vị trao đổi được khóa với đối tác dùng phần mềm khác.
- **A và B, tính bí mật** — bản mã giải được đúng từng byte theo cả hai chiều. Đây là yêu cầu tối thiểu của liên thông.
- **A và B, tính xác thực** — chữ ký số được xác minh đúng theo cả hai chiều, nên người nhận biết được thư đúng là do ai gửi.
- **C, chữ ký tách rời** — phục vụ trường hợp cần ký một tệp mà không thay đổi nội dung tệp đó.
- **D, tính toàn vẹn** — phép thử này chứng minh một **thuộc tính an toàn tiêu cực**: hệ thống *từ chối* dữ liệu đã bị sửa thay vì âm thầm cho ra bản rõ sai. Với hệ thống mật mã, thất bại đúng cách quan trọng ngang thành công đúng cách: một hệ thống

trả về bản rõ sai mà không báo lỗi còn nguy hiểm hơn một hệ thống không hoạt động.

B. ĐO HIỆU NĂNG

Lệnh chạy lại:

```
cd mailvelope
node scripts/benchmark-crypto.mjs
```

Mỗi giá trị là **trung vị** của nhiều lần chạy, để một lần chạy bất thường không làm lệch kết quả. Kịch bản kiểm tra tính đúng đắn dẫn sau mỗi vòng (độ dài dữ liệu sau khi giải mã phải khớp, chữ ký phải xác minh được) nên số liệu không thể đến từ một phép tính bị bỏ dờ.

B.1. Sinh cặp khóa

Bảng B.1. Thời gian sinh cặp khóa (thư viện thuần, một luồng)

Loại khóa	Trung vị	Các lần đo	Số lần chạy
RSA-2048	428 ms	564 / 428 / 362 ms	3
RSA-4096 (mặc định của sản phẩm)	2.785 ms	3.153 / 1.708 / 2.785 ms	3
ECC Curve25519	306 ms	348 / 254 / 306 / 318 / 247 ms	5

Độ tản mạn lớn của RSA là bình thường: thuật toán phải thử ngẫu nhiên cho tới khi tìm được số nguyên tố đủ lớn, nên thời gian phụ thuộc may rủi.

Đo bổ sung trên chính giao diện của bản phát hành, bằng kịch bản scripts/capture-workflow.mjs chạy trong trình duyệt thật:

Bảng B.2. Thời gian sinh khóa RSA-4096 qua giao diện thật, đo từ lúc bấm nút

Lần	Thời gian
Cặp khóa thứ nhất	10.237 ms (10,2 giây)
Cặp khóa thứ hai	8.116 ms (8,1 giây)

Con số này lớn hơn phép đo thư viện thuần vì bao gồm cả dựng giao diện, truyền tin giữa các thành phần của tiện ích và ghi vào kho lưu trữ. Đây là **thời gian người dùng thực sự chờ**, và là thao tác chỉ thực hiện **một lần** khi thiết lập ban đầu.

B.2. Mã hóa và giải mã

Mọi phép đo đều **mã hóa kèm ký số** và **giải mã kèm xác minh chữ ký** — đúng cấu hình mặc định của sản phẩm, không phải chỉ mã hóa đơn thuần.

Bảng B.3. Thời gian xử lý và độ giãn nở bản mã

Dữ liệu	Mã hóa + ký	Giải mã + xác minh	Kích thước bản mã	Độ giãn nở	Số lần chạy
Thư tiếng Việt tiêu biểu (~5 KB)	36,3 ms	19,4 ms	~9,3 KB	+87,8%	7
1 KB	14,9 ms	19,2 ms	3,0 KB	+197,1%	7
10 KB	16,3 ms	14,7 ms	15,5 KB	+51,8%	7
100 KB	34,0 ms	46,9 ms	141 KB	+37,2%	5
1 MB	115,1 ms	25,9 ms	1,36 MB	+35,7%	5
5 MB	403,5 ms	86,7 ms	6,78 MB	+35,6%	3

B.2.1. Cách đọc bảng này

Về độ trễ. Với thư điện tử thông thường, độ trễ do mã hóa nằm trong khoảng **15–36 mili giây** — dưới ngưỡng người dùng cảm nhận được (khoảng 100 ms). Người dùng không thấy sản phẩm chậm đi. Với tệp đính kèm 5 MB, tổng thời gian dưới nửa giây.

Về độ giãn nở. Bản mã lớn hơn bản rõ khoảng **36%** với dữ liệu lớn. Nguyên nhân chính là mã hóa ASCII-armor (Base64), tự nó đã làm tăng 33%, cần thiết để khối bản mã đi qua được hệ thống thư điện tử vốn chỉ bảo đảm truyền văn bản. Với dữ liệu nhỏ, tỷ lệ cao hơn vì phần tiêu đề cố định (khóa phiên được bọc bằng RSA-4096 cho mỗi người nhận, khối chữ ký) chiếm tỷ trọng lớn — nhưng giá trị tuyệt đối vẫn nhỏ.

Vì sao đây là trường hợp xấu nhất. Các phép đo theo kích thước dùng **dữ liệu ngẫu nhiên không nén được**. OpenPGP nén dữ liệu trước khi mã hóa; văn bản thật nén được đáng kể, nên độ giãn nở thực tế với thư văn bản thấp hơn con số trong bảng. Dùng dữ liệu ngẫu nhiên là lựa chọn có chủ đích: số liệu phản ánh giới hạn trên chứ không phải một kết quả đẹp nhờ tính chất của dữ liệu thử.

Hệ quả thực tế cần lưu ý khi triển khai. Hạn mức một bức thư của Gmail là 25 MB. Với độ giãn nở ~36%, tệp đính kèm gốc lớn nhất gửi được là khoảng **18 MB**. Cần đưa điều này vào hướng dẫn người dùng.

Vì sao giải mã đôi khi nhanh hơn mã hóa. Với dữ liệu lớn, mã hóa phải nén, sinh khóa phiên, bọc khóa phiên bằng RSA cho từng người nhận và tạo chữ ký; giải mã chỉ mở một khóa phiên rồi giải mã đối xứng. Chênh lệch là hợp lý, không phải dấu hiệu bất thường.

C. HÌNH ẢNH GIAO DIỆN

Mọi ảnh chụp trong hồ sơ đều được sinh bằng kịch bản nạp **chính gói phát hành** vào trình duyệt Chrome ở chế độ không giao diện, rồi chụp lại. Không có ảnh nào là bản dựng minh họa hay ảnh chỉnh sửa.

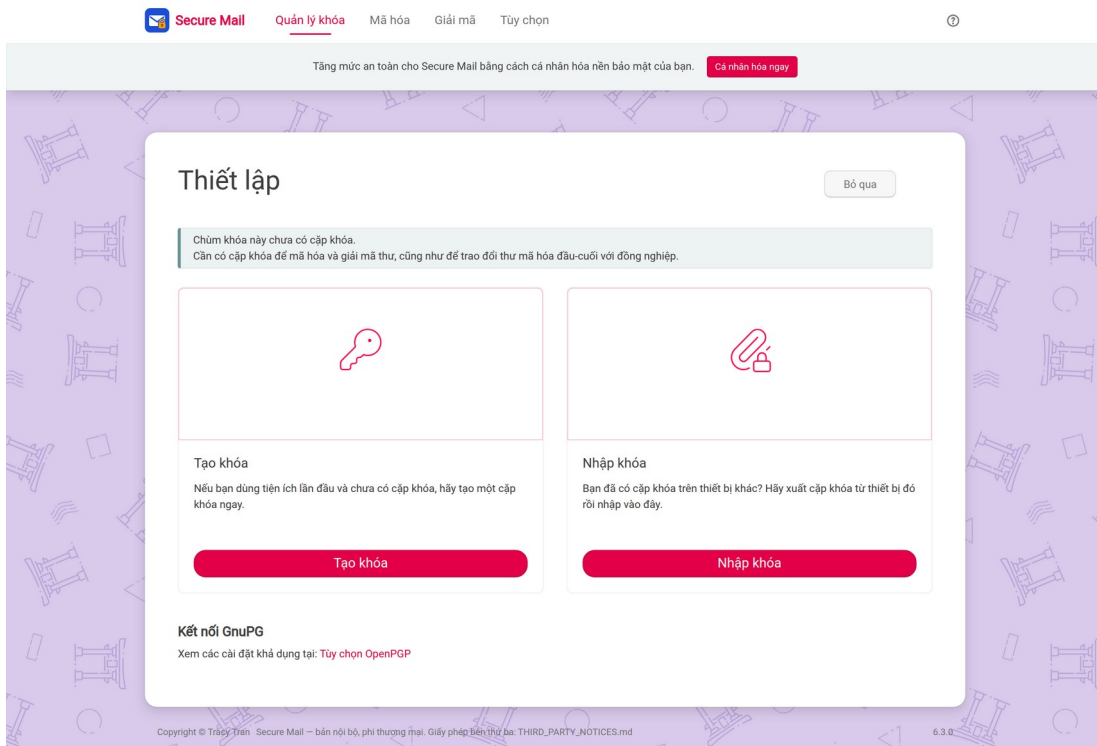
Lệnh chạy lại:

```
cd mailvelope
node scripts/capture-screenshots.mjs # các màn hình tĩnh, tiếng Anh và tiếng Việt
node scripts/capture-workflow.mjs # quy trình đầy đủ: sinh khóa thật rồi chụp
```

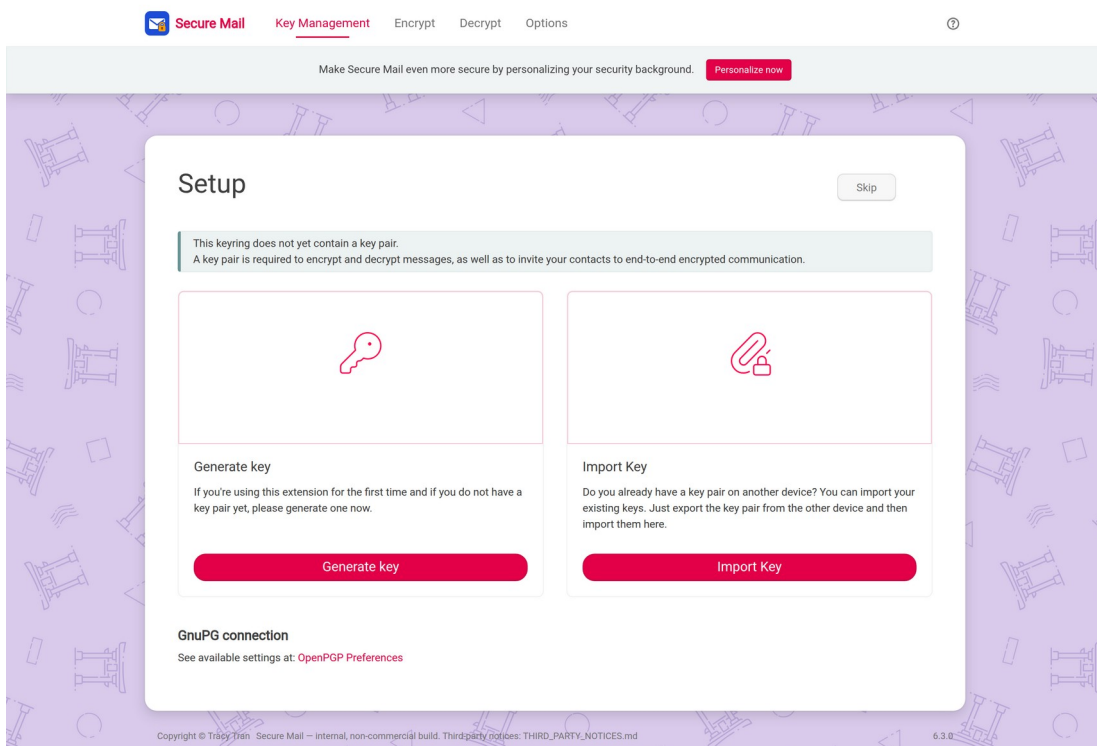
Kịch bản thứ hai thực hiện trọn quy trình: sinh hai cặp khóa RSA-4096 thật qua giao diện, mở màn hình chùm khóa, mở màn hình mã hóa và giải mã, đo thời gian từng bước. Vì vậy ảnh chụp chùm khóa cho thấy **khóa thật**, không phải dữ liệu dựng sẵn.

Bảng C.1. Danh mục ảnh chụp kèm theo hồ sơ

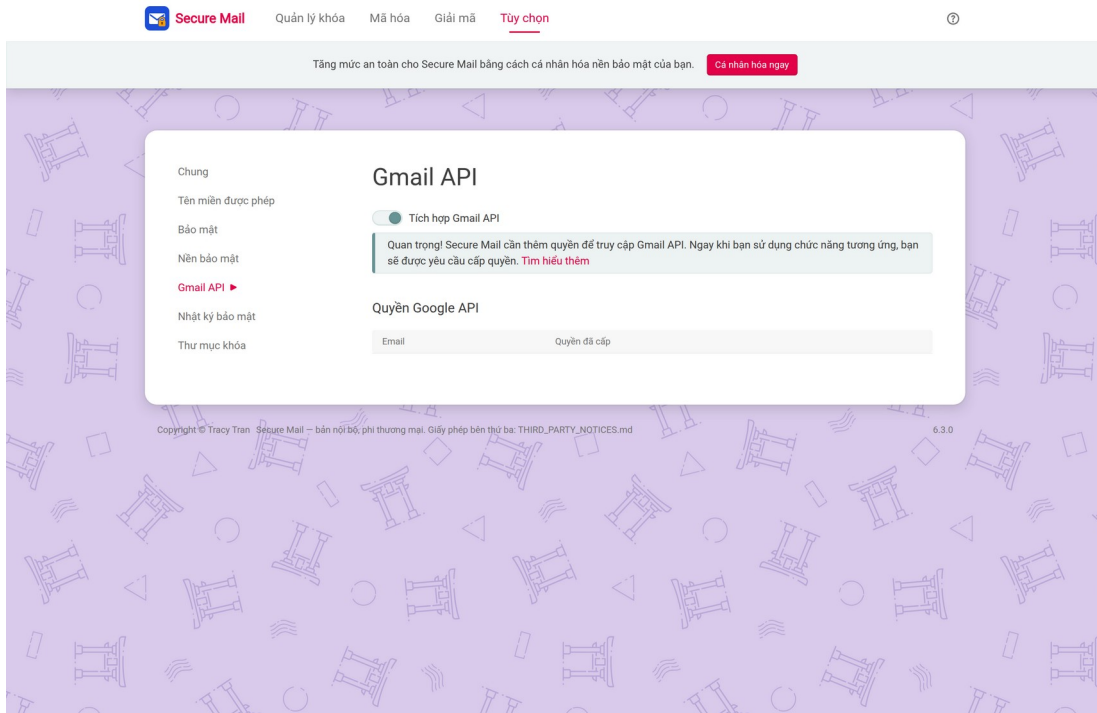
Tệp	Nội dung
settings-general.{en,vi}.png	Trang Tùy chọn → Chung, có bộ chọn ngôn ngữ
settings-keyserver.{en,vi}.png	Thư mục khóa — bốn cơ chế tra cứu bên ngoài ở trạng thái tắt
settings-security.{en,vi}.png	Tùy chọn bảo mật
settings-provider.{en,vi}.png	Cấu hình Gmail API
settings-watchlist.{en,vi}.png	Danh sách tên miền được phép
onboarding.{en,vi}.png	Màn hình thiết lập ban đầu
key-generate.{en,vi}.png	Màn hình tạo khóa
keyring-display.{en,vi}.png	Màn hình quản lý khóa
wf-01...wf-09	Chín ảnh của quy trình đầy đủ với khóa thật



Hình C.1. Màn hình thiết lập ban đầu bằng tiếng Việt, có nêu nguyên tắc quản trị viên chỉ giữ khóa công khai



Hình C.2. Cùng màn hình đó khi người dùng chọn tiếng Anh — cùng một bản cài đặt, chỉ khác lựa chọn ngôn ngữ



Hình C.3. Cấu hình Gmail API. Bản gốc hiển thị thêm bằng giấy phép thương mại cho tài khoản Workspace; bảng đó đã được gỡ bỏ

D. HƯỚNG DẪN TÁI LẬP TOÀN BỘ

Trình tự dưới đây dựng lại sản phẩm từ mã nguồn và chạy lại mọi phép kiểm chứng.

```
# 1. Chuẩn bị (Node.js >= 24 đề' biên dịch)
cd mailvelope
npm ci

# 2. Tạo tệp bí mật OAuth cục bộ (không nằm trong kho mã nguồn)
cp src/modules/oauth.local.example.js src/modules/oauth.local.js
# rồi điền client secret của dự án Google Cloud của đơn vị

# 3. Biên dịch grunt bản phát hành
npx grunt prod

# 4. Chạy toàn bộ kiểm thử' và kiểm toán
npx jest --selectProjects unit # 449 phép thử'
node scripts/verify-translation.mjs # kiểm toán bản dịch
./scripts/verify-no-external-endpoints.sh build/chrome
node scripts/interop-gnupg.mjs # cần cài GnuPG
node scripts/benchmark-crypto.mjs # đo hiệu năng

# 5. Chụp lại ảnh giao diện (cần Chrome for Testing)
npx puppeteer browsers install chrome
node scripts/capture-screenshots.mjs
node scripts/capture-workflow.mjs
```

```
# 6. Kiểm thử' bản di động
cd ../mobile
npm ci
npx jest # 68 phép thử'
./scripts/verify-desktop-untouched.sh

# 7. Dựng lại toàn bộ hình vẽ và tài liệu của hồ sơ
cd ../docs-sangkien/tools
python3 make_diagrams.py # .drawio + .svg
+ .png
python3 build_docx.py ../02-THUYET-MINH-SANG-KIEN.md \
  ../02-THUYET-MINH-SANG-KIEN.docx ..
```

Lưu ý về việc nạp tiện ích để chụp ảnh. Bản Google Chrome thương mại từ chối tham số `--load-extension` (báo *"not allowed in Google Chrome, ignoring"*). Phải dùng bản **Chrome for Testing**, cài bằng `npx puppeteer browsers install chrome`. Kịch bản chụp ảnh đã ghi rõ điều này trong phần chú thích.

E. TỰ KIỂM CHỨNG SỐ LIỆU CỦA CHÍNH HỒ SƠ

Một hồ sơ yêu cầu Hội đồng tin vào các con số của nó thì phải chứng minh được các con số đó khi được hỏi. Vì vậy hồ sơ này đi kèm một kịch bản **tự kiểm tra chính mình**: nó đọc lại từng con số nêu trong tài liệu, tính lại đại lượng tương ứng từ kho mã nguồn, từ gói phát hành hoặc từ tệp kết quả đo, rồi báo sai nếu hai bên không khớp.

Lệnh chạy lại:

```
cd docs-sangkien
python3 tools/verify_claims.py # đối chiếu 33 số' liệu
python3 tools/check_consistency.py # đánh số' và tham chiếu chéo
```

Kết quả:

```
33 phép kiểm tra, 33 đạt, 0 sai
Không phát hiện vấn đề' về' đánh số' và tham chiếu.
```

Kịch bản thứ nhất đối chiếu: số mục dịch, độ phủ bản dịch, số ngôn ngữ, số dòng danh mục ngôn ngữ, kết quả hai bộ kiểm thử, kết quả liên thông và phiên bản GnuPG dùng đối chứng, kết quả kiểm toán bề mặt mạng, sự vắng mặt của từng điểm cuối đã gỡ trong gói phát hành, phiên bản thư viện mật mã, việc gói phụ thuộc đo lường đã bị gỡ, năm mốc thời gian mã hóa, hai lần đo sinh khóa trên giao diện thật, mã định danh tiện ích, mã băm gói phát hành, số tệp trong gói, số dòng các kịch bản, số tệp dự án di

động, số tài liệu vận hành, số tệp biểu tượng, sự tồn tại của mọi hình được viện dẫn, và việc mỗi sơ đồ có đủ ba định dạng.

Kịch bản thứ hai kiểm tra bảng và hình được đánh số liên tiếp, mọi tham chiếu "Bảng N", "Hình N", "mục N" đều trở tới một mục có thật, kể cả tham chiếu chéo từ Bản thuyết minh sang Phụ lục này.

Toàn văn kết quả: `assets/measurements/claims-audit.txt` và `assets/measurements/consistency-audit.txt`.

Vì sao đưa việc này vào hồ sơ. Một tài liệu dài, nhiều số liệu, qua nhiều lần sửa thì gần như chắc chắn có chỗ lệch: một bảng được cập nhật mà đoạn văn nhắc tới nó thì không, một hình được đánh số lại mà tham chiếu thì quên. Những lỗi đó nhỏ nhưng làm hỏng lòng tin vào toàn bộ phần còn lại. Cách xử lý ở đây giống hết cách xử lý các khẳng định về an toàn trong sản phẩm: **biến việc kiểm tra thành một lệnh chạy được, thay vì dựa vào việc đọc kỹ.**

F. TẬP DỮ LIỆU KÈM THEO

Bảng F.1. Tập kết quả gốc kèm theo hồ sơ

Tệp	Nội dung
<code>assets/measurements/crypto-benchmark.json</code>	Toàn bộ số liệu hiệu năng, kèm cấu hình máy đo và từng lần đo
<code>assets/measurements/interop-gnupg.json</code>	Kết quả 8 phép thử liên thông, dạng máy đọc được
<code>assets/measurements/translation-audit.json</code>	Báo cáo kiểm toán bản dịch, kèm chi tiết từng nhóm lỗi
<code>assets/measurements/workflow-timings.json</code>	Thời gian đo trên giao diện thật
<code>assets/measurements/network-audit.txt</code>	Toàn văn kết quả kiểm toán bề mặt mạng
<code>assets/measurements/interop-gnupg.txt</code>	Toàn văn nhật ký phép thử liên thông
<code>assets/measurements/translation-audit.txt</code>	Toàn văn kết quả kiểm toán bản dịch
<code>assets/measurements/desktop-tests.txt</code>	Kết quả kiểm thử bản máy tính
<code>assets/measurements/mobile-tests.txt</code>	Kết quả kiểm thử bản di động
<code>assets/measurements/claims-audit.txt</code>	Kết quả tự kiểm chứng 33 số liệu của hồ sơ

assets/measurements/consistency-audit.txt	Kết quả kiểm tra đánh số và tham chiếu chéo
assets/diagrams/*.drawio	Sơ đồ nguồn, mở và sửa được bằng diagrams.net
assets/diagrams/*.svg	Sơ đồ dạng vector
assets/diagrams/*.png	Sơ đồ dạng ảnh, dùng để nhúng vào tài liệu
assets/screenshots/*.png	Ảnh chụp giao diện

Mỗi sơ đồ có đủ **ba định dạng từ cùng một nguồn định nghĩa** (tools/make_diagrams.py), nên tệp sửa được và ảnh xuất bản không bao giờ lệch nhau. Nếu cần chỉnh sửa một hình, mở tệp .drawio tương ứng bằng diagrams.net.